torch.matmul#

https://pytorch.org/docs/stable/generated/torch.matmul.html#torch.matmul

Description:

Matrix product of two tensors. The behavior depends on the dimensionality of the tensors as follows: If both tensors are 1-dimensional, the dot product (scalar) is returned. If both arguments are 2-dimensional, the matrix-matrix product is returned. If the first argument is 1-dimensional and the second argument is 2-dimensional, a 1 is prepended to its dimension for the purpose of the matrix multiply. After the matrix multiply, the prepended dimension is removed.

Function Signature

```
torch.matmul(input, other, *, out=None) → Tensor#
Parameters
Keyword Arguments
```

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> # vector x vector
>>> tensor1 = torch.randn(3)
>>> tensor2 = torch.randn(3)
>>> torch.matmul(tensor1, tensor2).size()
torch.Size([])
>>> # matrix x vector
>>> tensor1 = torch.randn(3, 4)
>>> tensor2 = torch.randn(4)
>>> torch.matmul(tensor1, tensor2).size()
torch.Size([3])
>>> # batched matrix x broadcasted vector
>>> tensor1 = torch.randn(10, 3, 4)
>>> tensor2 = torch.randn(4)
>>> torch.matmul(tensor1, tensor2).size()
torch.Size([10, 3])
>>> # batched matrix x batched matrix
>>> tensor1 = torch.randn(10, 3, 4)
>>> tensor2 = torch.randn(10, 4, 5)
>>> torch.matmul(tensor1, tensor2).size()
torch.Size([10, 3, 5])
>>> # batched matrix x broadcasted matrix
>>> tensor1 = torch.randn(10, 3, 4)
>>> tensor2 = torch.randn(4, 5)
>>> torch.matmul(tensor1, tensor2).size()
torch.Size([10, 3, 5])
```

Notes & Warnings

⚠ WARNING Warning Sparse support is a beta feature and some layout(s)/dtype/device combinations may not be supported, or may not have autograd support. If you notice missing functionality please open a feature request.

NOTE Note The 1-dimensional dot product version of this function does not support an out parameter.

Detailed Documentation

Documentation

Matrix product of two tensors.

The behavior depends on the dimensionality of the tensors as follows:

If both tensors are 1-dimensional, the dot product (scalar) is returned.

If both arguments are 2-dimensional, the matrix-matrix product is returned.

If the first argument is 1-dimensional and the second argument is 2-dimensional, a 1 is prepended to its dimension for the purpose of the matrix multiply. After the matrix multiply, the prepended dimension is removed.

If the first argument is 2-dimensional and the second argument is 1-dimensional, the

matrix-vector product is returned.

If both arguments are at least 1-dimensional and at least one argument is N-dimensional (where N > 2), then a batched matrix multiply is returned. If the first argument is 1-dimensional, a 1 is prepended to its dimension for the purpose of the batched matrix multiply and removed after. If the second argument is 1-dimensional, a 1 is appended to its dimension for the purpose of the batched matrix multiply and removed after.

The first N-2 dimensions of each argument, the batch dimensions, are broadcast (and thus must be broadcastable). The last 2, the matrix dimensions, are handled as in the matrix-matrix product.

For example, if input is a (j×1×n×m)(j \times 1 \times n \times m)(j×1×n×m) tensor and other is a (k×m×p)(k \times m \times p)(k×m×p) tensor, the batch dimensions are (j×1)(j \times 1)(j×1) and (k)(k)(k), and the matrix dimensions are (n×m)(n \times m)(n×m) and (m×p)(m \times p)(m×p). out will be a (j×k×n×p)(j \times k \times n \times p)(j×k×n×p) tensor.

This operation has support for arguments with sparse layouts. In particular the matrix-matrix (both arguments 2-dimensional) supports sparse arguments with the same restrictions as torch.mm()

Sparse support is a beta feature and some layout(s)/dtype/device combinations may not be supported, or may not have autograd support. If you notice missing functionality please open a feature request.

This operator supports TensorFloat32.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

The 1-dimensional dot product version of this function does not support an out parameter.

input (Tensor) – the first tensor to be multiplied

other (Tensor) – the second tensor to be multiplied

out (Tensor, optional) – the output tensor.
